

Figure 7.3 Tokens produced by invoking the _analyze endpoint

EXPLICIT ANALYZER TESTS

In listing 7.1, we didn't mention the analyzer, although the engine applied the `standard` analyzer by default. However, we can also explicitly enable an analyzer. The code in the following listing enables the `simple` analyzer.

Listing 7.2 Explicitly enabling an analyzer

```
GET _analyze
{
  "text": "James Bond 007",
  "analyzer": "simple"
}
```

The `simple` analyzer (we learn about various types of analyzers in the next section) truncates text when a non-letter character is encountered. So this code produces only two tokens: "james" and "bond" ("007" is truncated), as opposed to the three tokens produced by the earlier script that used the `standard` analyzer.

If you are curious, change the analyzer to `english`. The output tokens are then "jame", "bond", and "007". The notable point is that "james" is stemmed to "jame" when the `english` analyzer is applied. (We discuss the `english` analyzer in section 7.3.7.)

CONFIGURING ANALYZERS ON THE FLY

We can also use the `_analyze` API to mix and match filters and tokenizers, essentially creating a custom analyzer on the fly (we are not really building or developing a new analyzer as such). An on-demand custom analyzer is shown in the following code.